

Lecture 1

1. Introduction to Machine Learning

Machine Learning is a method that enables computers to learn patterns from data and make predictions or decisions without being explicitly programmed for every situation.

Traditional programming:

Data+Rules→Output

Machine Learning:

Data+Expected Outputs→Learned Model

Example

Problem: Identify whether an email is spam.

The model learns from previous emails containing:

- Email text
- Sender information
- Number of links
- Suspicious words
- Spam or not-spam label

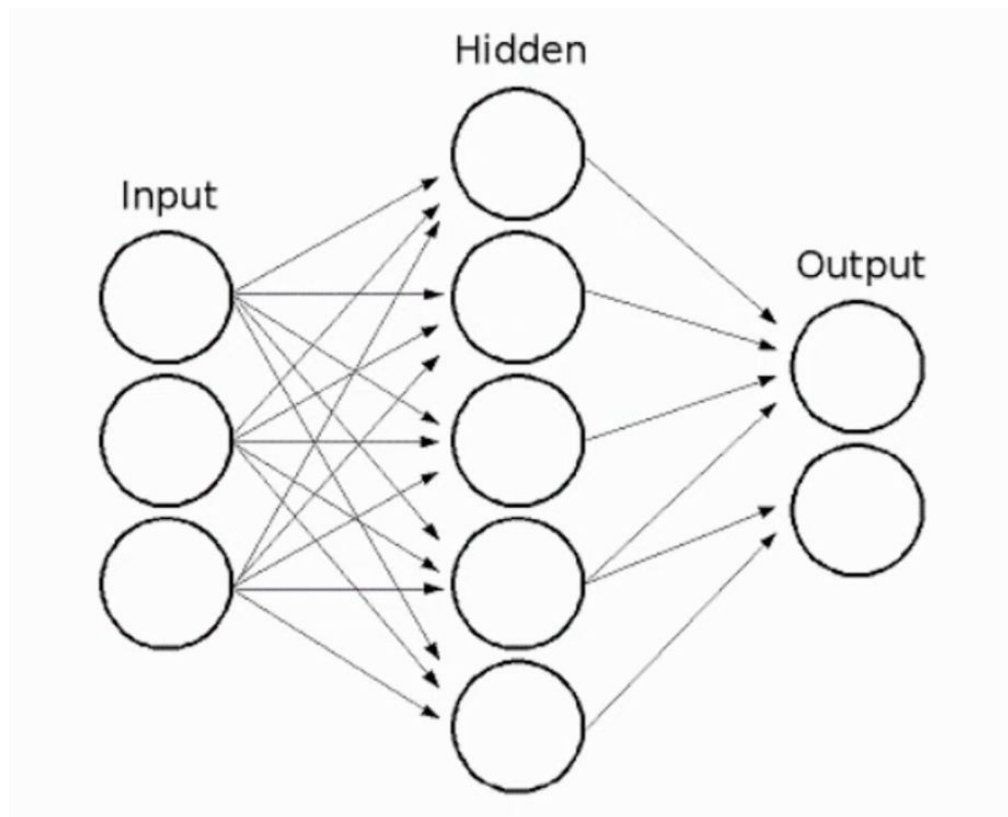
After training, the model predicts whether a new email is spam

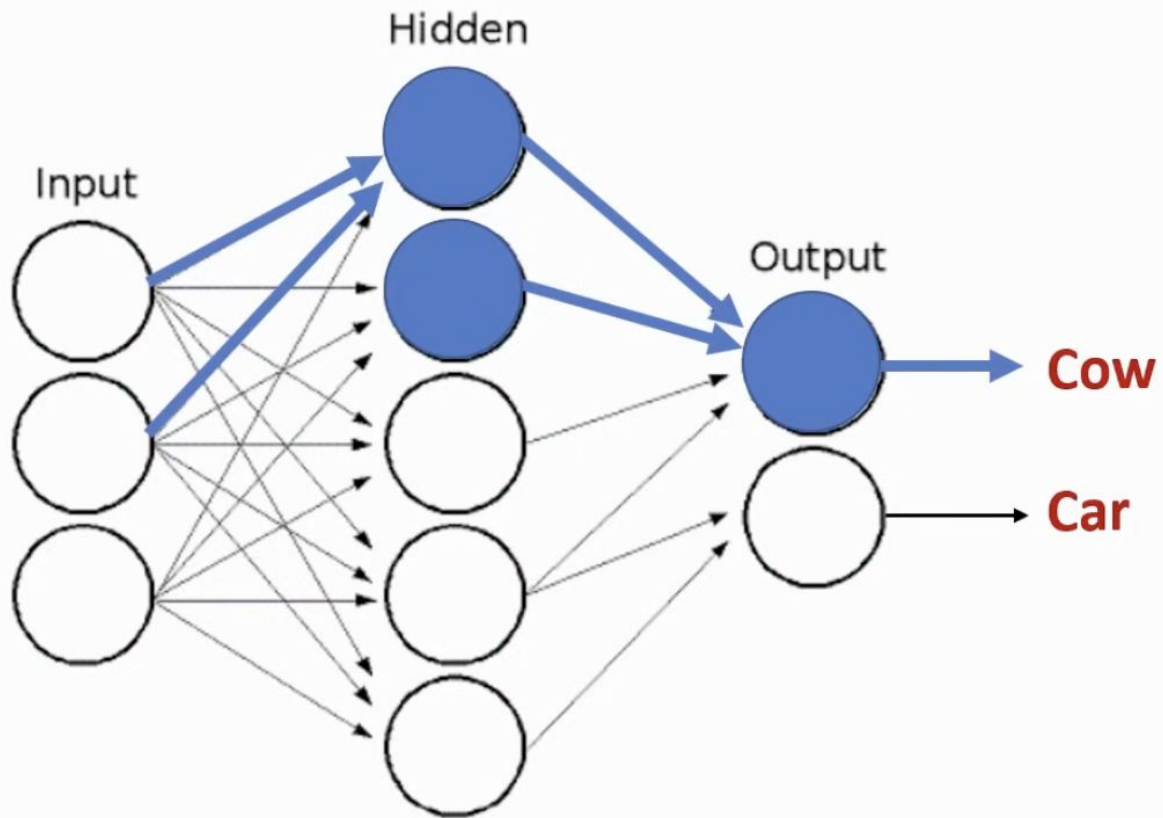
Why Do We Need Machine Learning?

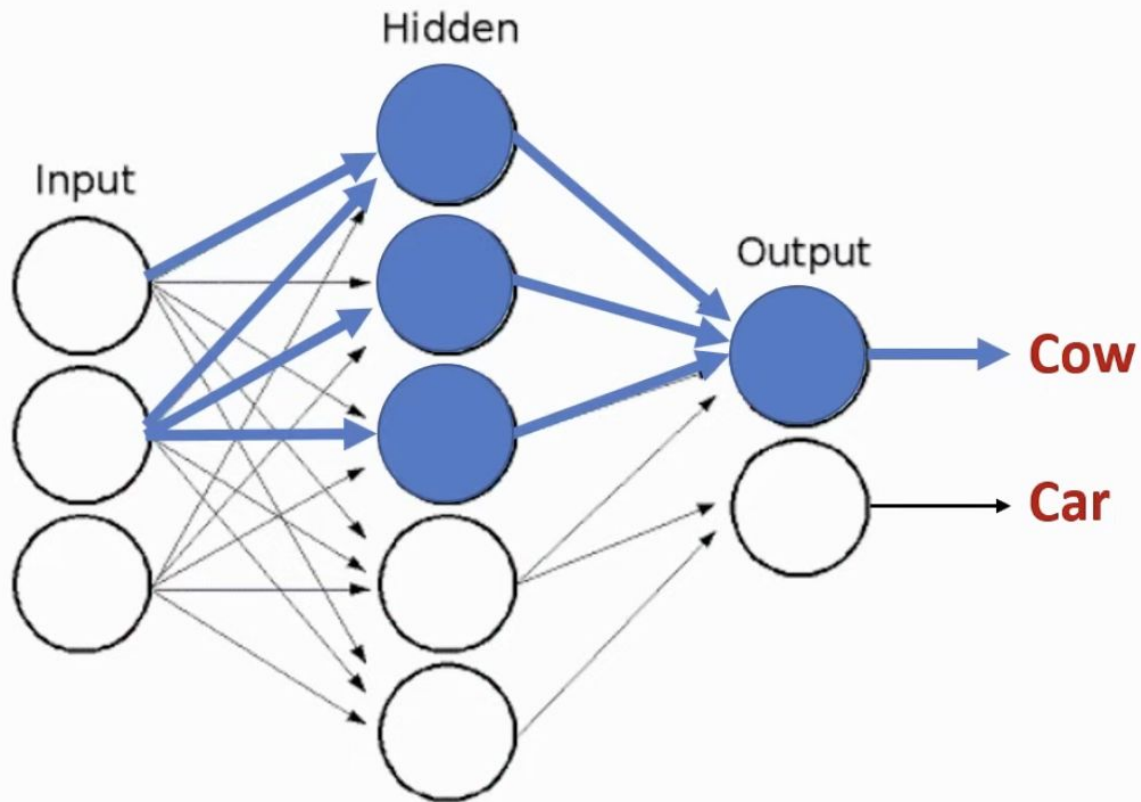
Traditional programs become difficult when:

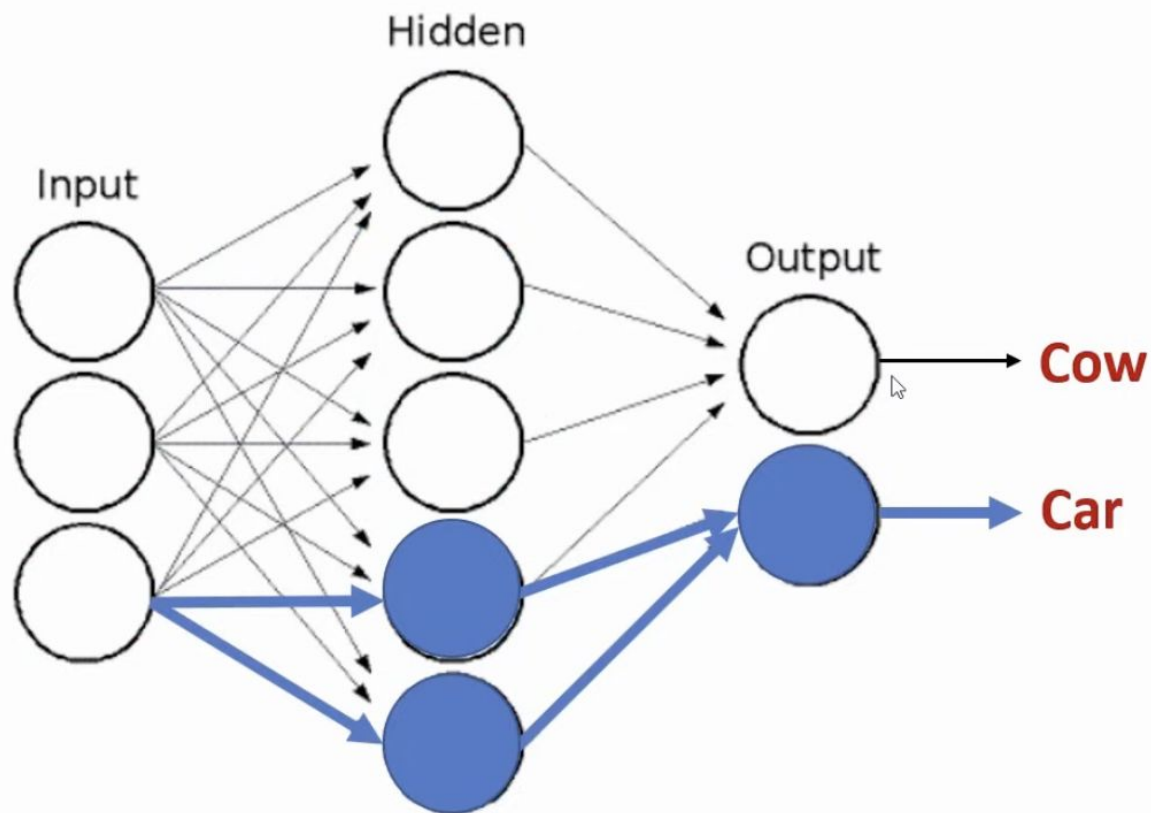
- The rules are too complex
- The amount of data is very large
- Patterns change over time
- Human-level recognition is required
- Predictions must be made automatically

Examples: Face recognition, fraud detection and medical diagnosis.









AI, ML and Deep Learning

Artificial Intelligence

The broader field of creating intelligent computer systems.

Machine Learning

A subset of AI in which systems learn from data.

Deep Learning

A subset of ML that uses multi-layer neural networks.

Deep Learning $\subset$ Machine Learning $\subset$ AI

2. Types of Machine Learning

There are three major types:

1. **Supervised Learning**
Learns from labelled data.
2. **Unsupervised Learning**
Finds hidden patterns in unlabelled data.
3. **Reinforcement Learning**
Learns through rewards and penalties.

Supervised Learning

Supervised Learning uses input data with known answers.

Main tasks:

- Classification
- Regression

Examples:

- Spam or not spam
- Disease detection
- House price prediction
- Student result prediction

Supervised Learning

2.1 Supervised Learning

Supervised learning trains a model using labeled examples. Each input X is paired with a correct target y . The model learns a function $f(X)$ approximately equal to y .

Training data: $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

- Regression predicts a continuous number, such as price or temperature.
- Classification predicts a category, such as pass/fail or spam/not spam.

What is Regression?

Regression is a supervised machine learning technique used to predict a **continuous numerical value**.

Examples:

- House price: ₺50,00,000
- Temperature: 31.5°C
- Student marks: 78.5
- Monthly sales: ₺2,50,000
- Delivery time: 35 minutes

The output is a number, not a category.

Suppose we have the following data:

Here:

- **Feature/Input (X):** Study hours
- **Target/Output (Y):** Exam marks

The model learns the relationship between study hours and exam marks. It can then predict the marks of a student who studies for 6 hours.

Study Hours (X)	Exam Marks (Y)
-----------------	----------------

1	40
---	----

2	48
---	----

3	55
---	----

4	65
---	----

5	72
---	----

In reinforcement learning, an agent interacts with an environment. At each step it observes a state, chooses an action, receives a reward, and moves to another state. It learns a policy that maximizes long-term cumulative reward.

Term	Meaning
Agent	The learner or decision-maker.
Environment	The world in which the agent acts.
State (s)	Current situation.
Action (a)	A choice made by the agent.
Reward (r)	Immediate feedback after an action.
Policy $\pi(a s)$	Rule for choosing actions in states.
Episode	A sequence from a starting state to a terminal state.

Goal: maximize expected cumulative reward $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$

Here γ (0 to 1) is the discount factor. A small γ emphasizes immediate reward; a large γ values future reward more strongly.

Classification vs Regression

Features	Regression	Classification
Output Type	Predicts a continuous numeric value	Predicts a categorical label
Goal	Fitting the best-fit line or curve	Drawing a decision boundary between classes
Error Measure	Uses metrics like MSE, MAE, RMSE	Uses metrics like Accuracy, Precision, Recall, F1 Score
Use Cases	Forecasting, price prediction, risk estimation	Image recognition, sentiment analysis, fraud detection
Handling Outliers	Highly sensitive	Less sensitive

Unsupervised Learning

Unsupervised Learning works with data that does not have predefined labels.

The model attempts to discover:

- Similar groups
- Hidden patterns
- Relationships
- Unusual observations

Example: Grouping customers based on purchasing behaviour.

Unsupervised Learning

2.2 Unsupervised Learning

Unsupervised learning uses data without target labels. Its purpose is to discover hidden structure, groups, relationships, or lower-dimensional representations.

- Clustering: K-Means groups similar customers.
- Dimensionality reduction: PCA compresses many features.
- Association: discovers items that frequently occur together.
- Anomaly detection: identifies unusual observations.

Reinforcement Learning

An agent learns by interacting with an environment.

- Correct action $\rightarrow$ Reward
- Incorrect action $\rightarrow$ Penalty
- Goal $\rightarrow$ Maximize total reward

Examples:

- Game-playing AI
- Robot navigation
- Self-driving systems
- Traffic signal control

Basic Machine Learning Terminology

Dataset -> Collection of data

Sample -> One observation or record

Feature -> Input variable

Label -> Expected output

Model -> Learned mathematical system

Training -> Process of learning from data

Prediction -> Output generated for new data

3. Features, Labels, Samples and Dataset

Types of Features:

1. Numerical: age, income, temperature. It may be continuous or discrete.
2. Categorical: department, blood group, city. It may be nominal or ordinal.
3. Binary: yes/no, 0/1.
4. Text, image, audio, and time-series data require suitable numerical representations.

1. A label is not available for new real-world samples; the model predicts it.
2. An ID column is usually an identifier, not a meaningful feature.
3. A categorical number such as department code 1, 2, 3 is not automatically a continuous quantity.
4. Using future information or the target itself as a feature causes data leakage.

Example Dataset

Student Result Prediction				
Study Hours	Attendance	Assignment Score	Result	
2	60%	45	Fail	
5	80%	70	Pass	
7	90%	85	Pass	

Features:

- Study Hours
- Attendance
- Assignment Score

Label: Result

Machine Learning Workflow

1. Define the problem
2. Collect the dataset
3. Explore and clean the data
4. Select features and labels
5. Split the dataset
6. Choose an algorithm
7. Train the model
8. Evaluate the model
9. Improve the model
10. Deploy and monitor

Training and Testing Data

A dataset is commonly divided into:

- **Training set:** Used to teach the model
- **Validation set:** Used to tune the model
- **Test set:** Used to evaluate final performance

Example division:

- Training: 70%
- Validation: 15%
- Testing: 15%

The test data should remain unseen during training.

Real-World Applications

Machine Learning is used in:

- Healthcare: Disease detection
- Banking: Fraud detection
- Education: Student performance prediction
- Agriculture: Crop disease detection
- E-commerce: Product recommendation
- Cybersecurity: Intrusion detection
- Transportation: Traffic prediction
- Entertainment: Movie recommendation

Challenges of Machine Learning

A Machine Learning model may fail because of:

- Insufficient data
- Incorrect or missing data
- Biased datasets
- Irrelevant features
- Overfitting
- Wrong algorithm selection
- Data privacy and security issues

Important: A model is only as reliable as its data and evaluation process.

4. Training, Validation and Test Set

4.1 Why Split the Dataset?

If a model is evaluated on the same examples used for training, it may simply memorize them. A separate dataset is required to estimate performance on unseen data. This ability is called generalization.

Term	Meaning
Training set	Used to estimate model parameters such as weights and coefficients.
Validation set	Used to compare models and tune hyperparameters.
Test set	Used only for the final, unbiased performance estimate.

Typical Ratios

- 70% training, 15% validation, 15% test.
- 80% training, 10% validation, 10% test.
- When using cross-validation, a common choice is 80% development/training data and 20% final test data.

Parameters vs Hyperparameters

Term	Meaning
Parameter	Learned from training data, e.g., linear-regression coefficients.
Hyperparameter	Chosen before/during tuning, e.g., K in KNN, tree depth, learning rate.

Stratification, Shuffling and Time Order

1. Stratified splitting preserves class proportions in classification.
2. Random shuffling reduces ordering bias for independent observations.
3. Time-series data must usually be split chronologically; future observations must not train a model evaluated on the past.
4. Samples from the same person/group should not be spread across train and test if that creates leakage.

5. Data Preprocessing

Data preprocessing converts raw, incomplete, inconsistent, or differently scaled data into a reliable numerical form suitable for a machine-learning algorithm. Good preprocessing often improves a model more than choosing a more complex algorithm.

Main Steps of Preprocessing

1. Inspect data types, distributions, ranges, duplicates, and missing values.
2. Correct invalid values and inconsistent formats.
3. Handle missing values.
4. Handle duplicates and, when justified, outliers.
5. Encode categorical variables.
6. Scale numerical features when required.
7. Split data correctly and use a pipeline to prevent leakage.

Practice

Customer ID	Age	Gender	City	Monthly Charge	Join Date	Churn
C01	25	Male	Dhaka	500	01-01-2024	No
C02	32	Female	Chittagong	700	2024/02/10	Yes
C03	32	female	Chittagong	700	10-Feb-2024	Yes
C03	32	female	Chittagong	700	10-Feb-2024	Yes
C04	150	Male	Dhaka	900	15-03-2024	No
C05	28	Female	dhaka		20-04-2024	No
C06		Male	Sylhet	650	25-05-2024	Yes
C07	40		Sylhet	10,000	01-06-2024	No

Cleaned Dataset

Customer ID	Age	Gender	City	Monthly Charge	Join Date	Churn
C01	25	Male	Dhaka	500	2024-01-01	No
C02	32	Female	Chittagong	700	2024-02-10	Yes
C03	32	Female	Chittagong	700	2024-02-10	Yes
C04	32	Male	Dhaka	900	2024-03-15	No
C05	28	Female	Dhaka	700	2024-04-20	No
C06	32	Male	Sylhet	650	2024-05-25	Yes
C07	40	Male	Sylhet	1,000	2024-06-01	No

Encoding

Customer ID	Age	Gender	City	Monthly Charge	Join Date	Churn
C01	25	Male	Dhaka	500	2024-01-01	0
C02	32	Female	Chittagong	700	2024-02-10	1
C03	32	Female	Chittagong	700	2024-02-10	1
C04	32	Male	Dhaka	900	2024-03-15	0
C05	28	Female	Dhaka	700	2024-04-20	0
C06	32	Male	Sylhet	650	2024-05-25	1
C07	40	Male	Sylhet	1,000	2024-06-01	0

City One-Hot Encoding

City	City_Chittagong	City_Dhaka	City_Sylhet
Chittagong	1	0	0
Dhaka	0	1	0
Sylhet	0	0	1

Dataset after encoding

City	City_Chittagong	City_Dhaka	City_Sylhet
Chittagong	1	0	0
Dhaka	0	1	0
Sylhet	0	0	1

Dataset after encoding

Age	Monthly Charge	Gender_Male	City_Chittagong	City_Dhaka	City_Sylhet	Churn
25	500	1	0	1	0	0
32	700	0	1	0	0	1
32	700	0	1	0	0	1
32	900	1	0	1	0	0
28	700	0	0	1	0	0
32	650	1	0	0	1	1
40	1,000	1	0	0	1	0

Scale numerical features

	Feature	Range
Numerical features may have very different ranges. For example:	Age	25–40
	Monthly Charge	500–1,000

Features with larger values may influence some machine-learning models more strongly. Therefore, scaling transforms numerical features into a similar range.

Scaling is commonly required for:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- K-Means Clustering
- Neural Networks

It is usually not required for tree-based models such as Decision Trees and Random Forests.

Min-Max Scaling

Min-Max Scaling transforms values into a range between 0 and 1.

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Where:

- x = original value
- $x_{\min}$ = minimum value of the feature
- $x_{\max}$ = maximum value of the feature
- x_{scaled} = scaled value

Min-Max Scaling

Example 1: Scaling Age

The minimum age is 25 and the maximum age is 40. For an age of 32:

$$\text{Scaled Age} = \frac{32 - 25}{40 - 25} = \frac{7}{15} = 0.47$$

Therefore, the scaled value of age 32 is approximately **0.47**.

Example 2: Scaling Monthly Charge

The minimum monthly charge is 500 and the maximum is 1,000. For a monthly charge of 700:

$$\text{Scaled Monthly Charge} = \frac{700 - 500}{1000 - 500} = \frac{200}{500} = 0.40$$

Therefore, the scaled value of monthly charge 700 is **0.40**.

Min-Max Scaling

Example 1: Scaling Age

The minimum age is 25 and the maximum age is 40. For an age of 32:

$$\text{Scaled Age} = \frac{32 - 25}{40 - 25} = \frac{7}{15} = 0.47$$

Therefore, the scaled value of age 32 is approximately **0.47**.

Example 2: Scaling Monthly Charge

The minimum monthly charge is 500 and the maximum is 1,000. For a monthly charge of 700:

$$\text{Scaled Monthly Charge} = \frac{700 - 500}{1000 - 500} = \frac{200}{500} = 0.40$$

Therefore, the scaled value of monthly charge 700 is **0.40**.

Dataset after encoding

Original Age	Scaled Age	Original Monthly Charge	Scaled Monthly Charge
25	0.00	500	0.00
32	0.47	700	0.40
32	0.47	700	0.40
32	0.47	900	0.80
28	0.20	700	0.40
32	0.47	650	0.30
40	1.00	1,000	1.00

The scaler must learn the minimum and maximum values only from the **training data**. The same learned values should then be used to transform the validation and test data. This prevents **data leakage**.

Data Preprocessing: Missing Values

1. Deletion: remove rows/columns only when missingness is limited and removal is justified.
2. Numerical imputation: mean is simple but sensitive to outliers; median is robust; model-based methods are possible.
3. Categorical imputation: mode or a separate Unknown category.
4. A missing-indicator feature can record whether the original value was absent.

Data Preprocessing: Categorical Encoding

Term	Meaning
Label/ordinal encoding	Maps ordered categories to numbers; use when order is meaningful.
One-hot encoding	Creates one binary column per category; suitable for nominal categories.
Target encoding	Uses target statistics; powerful but must be performed inside folds to avoid leakage.

Data Preprocessing: Feature Scaling

Min-Max scaling commonly maps values to $[0,1]$. Standardization produces approximately zero mean and unit variance. Scaling is important for KNN, SVM, PCA, logistic regression with regularization, and gradient-based models. Decision trees and random forests usually do not require it.

$$\text{Min-Max: } x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

$$\text{Standardization: } z = (x - \text{mean}) / \text{standard_deviation}$$

Data Preprocessing: Outliers

An outlier is an unusually distant observation. Detect it using domain rules, box plots/IQR, z-scores, or model-based methods. Do not delete every outlier automatically: it may be an error, a rare but valid event, or the most important case (for example, fraud).

IQR = Q3 - Q1; common bounds: Q1 - 1.5 IQR and Q3 + 1.5 IQR

Let,	$IQR = Q3 - Q1$	Lower Bound	Upper Bound:
$Q1 = 20$	$= 40 - 20$	$= Q1 - 1.5 \times IQR$	$= Q3 + 1.5 \times IQR$
$Q3 = 40$	$= 20$	$= 20 - 1.5 \times 20$	$= 40 + 1.5 \times 20$
		$= 20 - 30$	$= 40 + 30$
		$= -10$	$= 70$

6. Feature Engineering

Feature engineering is the process of creating, transforming, combining, or representing variables so that useful patterns become easier for a model to learn. It uses data knowledge, domain knowledge, and knowledge of the algorithm.

Feature engineering usually requires:

- Data knowledge: Understanding the data's format, distribution, and quality
- Domain knowledge: Understanding the real-world field related to the problem
- Algorithm knowledge: Understanding which types of features work well with a particular model

Feature Engineering

Suppose a sales dataset contains two columns:

Unit Price	Quantity
500	3

A new feature can be created by combining them:

$$\begin{aligned}\text{Total Price} &= \text{Unit Price} \times \text{Quantity} \\ &= 500 \times 3 \\ &= 1500\end{aligned}$$

The **Total Price** feature directly represents the total value of an order. It may help the model understand purchasing behaviour more easily than using **Unit Price** and **Quantity** separately.

Preprocessing vs. Feature Engineering

Data Preprocessing

Makes data clean and usable

Handles missing values

Handles duplicates and invalid values

Converts data into a model-compatible format

Feature Engineering

Makes data more informative

Creates meaningful new features

Combines or transforms existing features

Helps the model learn patterns more effectively

Examples

- Filling in missing ages → **Data preprocessing**
- Correcting an invalid date format → **Data preprocessing**
- Calculating age from date of birth → **Feature engineering**
- Calculating total price from price and quantity → **Feature engineering**

Some operations, such as encoding, scaling, and transformation, may be considered both preprocessing and feature engineering.

Common Techniques of Feature Engineering

- Interaction feature: price x quantity = total value.
- Ratio feature: debt / income.
- Polynomial feature: x^2 or $x_1 * x_2$ for nonlinear relationships.
- Date decomposition: year, month, weekday, hour, weekend flag.
- Binning: convert age into ranges; useful but may lose information.
- Log transformation: reduces strong right-skew and large-value influence.
- Aggregations: customer total purchases, mean transaction value, frequency.
- Text representation: bag-of-words, TF-IDF, embeddings.
- Cyclical encoding: sin/cos for hour or month where the end connects to the beginning.

Common Techniques of Feature Engineering

1. Interaction Features

An interaction feature represents the combined effect of two or more existing features.

For example:

$$\text{Total Value} = \text{Price} \times \text{Quantity}$$

Price	Quantity	Total Value
200	5	1,000
500	2	1,000

Although the price and quantity are different, both orders have the same total value. Therefore, `Total Value` expresses the financial importance of an order more directly.

Another example is:

$$\text{Study Interaction} = \text{Study Hours} \times \text{Attendance Rate}$$

This feature can help a model understand how study hours and attendance jointly affect examination performance.

Common Techniques of Feature Engineering

2. Ratio Features

A ratio feature represents the relationship between two numerical variables.

For example:

$$\text{Debt-to-Income Ratio} = \frac{\text{Debt}}{\text{Income}}$$

Monthly Income	Monthly Debt	Debt-to-Income Ratio
100,000	30,000	0.30
30,000	15,000	0.50

The first person has a higher amount of debt, but it represents only 30% of their income. The second person has less debt, but it represents 50% of their income.

Therefore, the ratio provides a clearer picture of financial pressure than debt or income alone.

Before creating a ratio feature, it is necessary to check whether the denominator can be zero.

Common Techniques of Feature Engineering

3. Polynomial Features

Polynomial feature engineering creates new features by calculating squares, cubes, or products of existing variables.

Examples include:

$$x^2, \quad x^3, \quad x_1x_2$$

Suppose the relationship between study hours and examination marks is not linear. Marks may improve rapidly at first, but the rate of improvement may decrease after many hours of study.

A polynomial regression model may use both x and x^2 :

$$\hat{y} = \beta_0 + \beta_1x + \beta_2x^2$$

This allows the model to learn a curved or nonlinear relationship.

However, creating too many polynomial features can increase model complexity and cause **overfitting**.

Common Techniques of Feature Engineering

4. Date Decomposition

Date decomposition means extracting meaningful components from a complete date or timestamp.

Suppose:

Order Date = 2026-09-07 15:30



The following features can be extracted:

Feature	Value
Year	2026
Month	9
Day	7
Day of the week	Monday
Hour	15
Weekend flag	0

These features can help the model learn time-related and seasonal patterns, such as:

- Whether sales increase on weekends
- Whether customers order more frequently in the evening
- Whether product demand changes during particular months

Common Techniques of Feature Engineering

5. Binning

Binning is the process of dividing a continuous numerical variable into several ranges or categories.

For example, exact ages can be converted into age groups:

Age	Age Group
15	Child/Teen
25	Young Adult
45	Middle-Aged
70	Senior

Binning can help a model identify group-based patterns. For example, customers within a particular age group may be more likely to purchase a specific product.

However, binning can cause information loss. If people aged 21 and 34 are both placed in the **Young Adult** group, the difference between their exact ages is no longer available to the model.

Common Techniques of Feature Engineering

6. Log Transformation

Some numerical features have many small values and a few extremely large values. This is known as a **right-skewed distribution**.

For example:

Income: 20,000; 25,000; 30,000; 40,000; 2,000,000



A log transformation can be applied:

$$x' = \log(1 + x)$$

Log transformation can:

- Reduce the influence of extremely large values
- Reduce skewness in the distribution
- Make the distribution more balanced
- Make certain relationships easier for a model to learn

The expression $\log(1 + x)$ is often used because it can handle zero:

$$\log(1 + 0) = 0$$

However, this transformation cannot be applied directly when the feature contains negative values.

Common Techniques of Feature Engineering

7. Aggregation Features

Aggregation features are created by summarising multiple records belonging to the same customer, product, machine, or group.

Suppose the raw transaction data is:

Customer	Purchase Amount
A	500
A	1,000
A	1,500

The following features can be created for Customer A:

Engineered Feature	Value
Total purchase amount	3,000
Average transaction value	1,000
Purchase frequency	3

These features directly represent the customer’s purchasing behaviour, spending power, and engagement.

Aggregation features must be created carefully. Future transactions should not be used to predict past behaviour, as this would cause **data leakage**.

Common Techniques of Feature Engineering

8. Text Representation

Most machine-learning models cannot directly process raw text. Therefore, text must be converted into numerical features.

Bag-of-Words

Bag-of-Words represents a document by counting how many times each word appears.

Text: "machine learning is useful"



machine	learning	is	useful
1	1	1	1

TF-IDF

Term Frequency–Inverse Document Frequency, or TF-IDF, assigns:

- Higher weights to words that are important in a particular document
- Lower weights to common words that appear in many documents

Embeddings

Embeddings convert words, sentences, or documents into dense numerical vectors.

Texts with similar meanings usually have vectors that are close to one another in the vector space.

Common Techniques of Feature Engineering

9. Cyclical Encoding

Some numerical features, such as hours, weekdays, and months, follow a repeating cycle.

For example:

- 23:00 and 00:00 are only one hour apart.
- December and January are consecutive months.
- Sunday is followed by Monday.

If hours are treated as ordinary numbers:

$$|23 - 0| = 23$$

The model may incorrectly assume that hour 23 and hour 0 are far apart, although they are actually adjacent.

Cyclical features can be encoded using sine and cosine functions:

$$\text{Hour}_{\sin} = \sin \left(\frac{2\pi \times \text{Hour}}{24} \right)$$

$$\text{Hour}_{\cos} = \cos \left(\frac{2\pi \times \text{Hour}}{24} \right)$$

Where:

- `Hour` represents an hour from 0 to 23
- `24` represents one complete daily cycle
- 2π represents one complete circle

Both sine and cosine values are required because sine alone may produce the same value for multiple hours. Together, they provide a unique circular position for every hour.

Common Techniques of Feature Engineering

9. Cyclical Encoding

Some numerical features, such as hours, weekdays, and months, follow a repeating cycle.

For example:

- 23:00 and 00:00 are only one hour apart.
- December and January are consecutive months.
- Sunday is followed by Monday.

If hours are treated as ordinary numbers:

$$|23 - 0| = 23$$

The model may incorrectly assume that hour 23 and hour 0 are far apart, although they are actually adjacent.

Cyclical features can be encoded using sine and cosine functions:

$$\text{Hour}_{\sin} = \sin\left(\frac{2\pi \times \text{Hour}}{24}\right)$$

$$\text{Hour}_{\cos} = \cos\left(\frac{2\pi \times \text{Hour}}{24}\right)$$

Where:

- `Hour` represents an hour from 0 to 23
- `24` represents one complete daily cycle
- 2π represents one complete circle

Both sine and cosine values are required because sine alone may produce the same value for multiple hours. Together, they provide a unique circular position for every hour.

Feature Engineering Example:

Suppose an e-commerce dataset contains the following raw features:

Date of Birth	Order Date	Unit Price	Quantity
1996-04-15	2026-09-06	500	3
1985-10-20	2026-09-07	1,200	2

The following new features can be created.

1. Customer Age

Customer age can be calculated from the date of birth and order date:

$$\text{Customer Age} = \text{Order Year} - \text{Birth Year}$$

For the first customer:

$$2026 - 1996 = 30$$

This is an approximate age. A more precise calculation would also compare the month and day.

2. Order Month

The month can be extracted from the order date:

2026-09-06 → September or 9



This feature may help identify monthly or seasonal purchasing patterns.

Feature Engineering Example:

3. Weekend Order

A binary feature can indicate whether an order was placed on a weekend:

- Saturday or Sunday → 1
- Any other day → 0

This feature helps the model determine whether customer behaviour changes during weekends.

4. Total Price

The total price can be calculated as:

$$\text{Total Price} = \text{Unit Price} \times \text{Quantity}$$

For the first order:

$$500 \times 3 = 1500$$

Final Engineered Dataset

Customer Age	Order Month	Weekend Order	Total Price
30	9	1	1,500
40	9	0	2,400

Raw dates may not be directly meaningful to a machine-learning model. Features such as Customer Age , Order Month , Weekend Order , and Total Price express useful real-world relationships more directly.

Feature Engineering Risks:

1. Leakage: using information unavailable at prediction time.
2. Overfitting: generating too many features for a small dataset.
3. Instability: ratios with a zero or very small denominator.
4. Loss of interpretability after complicated transformations.
5. Different transformations at training and prediction time.

Common Techniques of Feature Engineering

- Interaction feature: price x quantity = total value.
- Ratio feature: debt / income.
- Polynomial feature: x^2 or $x_1 * x_2$ for nonlinear relationships.
- Date decomposition: year, month, weekday, hour, weekend flag.
- Binning: convert age into ranges; useful but may lose information.
- Log transformation: reduces strong right-skew and large-value influence.
- Aggregations: customer total purchases, mean transaction value, frequency.
- Text representation: bag-of-words, TF-IDF, embeddings.
- Cyclical encoding: sin/cos for hour or month where the end connects to the beginning.

7. Feature Selection

Feature selection chooses a useful subset of existing features and removes irrelevant, redundant, noisy, or costly variables. Unlike feature extraction, it does not create entirely new combined dimensions.

- Reduces training time and storage.
- May improve generalization by reducing noise.
- Makes a model easier to interpret.
- Reduces multicollinearity and the curse of dimensionality.
- Can lower data-collection cost.

Feature Selection: Three Main Families

Term	Meaning
Filter methods	Model-independent statistical scores: correlation, chi-square, ANOVA, mutual information, variance threshold.
Wrapper methods	Train models on feature subsets: forward selection, backward elimination, recursive feature elimination (RFE).
Embedded methods	Selection occurs during training: Lasso, decision-tree importance, regularized models.

Feature Selection: Filter Methods

- Correlation measures linear association between numerical variables. Very high correlation among predictors suggests redundancy.
- Chi-square tests association between non-negative categorical/count features and a categorical target.
- ANOVA F-test compares variation between class groups with variation within groups.
- Mutual information can detect more general dependency, including nonlinear relationships.
- Variance threshold removes near-constant features but does not consider the target.

Feature Selection: Wrapper Methods

Wrapper methods can find feature interactions but are computationally expensive and may overfit the validation process. Embedded methods balance speed and model relevance. Lasso can shrink some coefficients exactly to zero; tree-based importance estimates how much features improve splits.

Feature Selection: Wrapper Methods

In embedded methods, model training and feature selection occur as parts of the same process.

Unlike filter methods, embedded methods are not model-independent. Unlike wrapper methods, they do not usually need to evaluate a large number of separate feature subsets.

Advantages

- Faster than wrapper methods.
- Considers the model's learning process while selecting features.
- Provides a balance between predictive relevance and computational efficiency.

Limitations

- The selected features depend on the specific model being used.
- Changing the model may result in a different set of selected features.
- Some feature-importance scores must be interpreted carefully.

Comparison of Filter, Wrapper and Embedded Methods

Property	Filter	Wrapper	Embedded
Selection basis	Statistical score	Model performance	Model training process
Model dependency	Independent	Dependent	Dependent
Computational cost	Low	High	Medium
Feature interactions	Usually ignored	Can detect	Can often detect
Overfitting risk	Relatively low	Higher	Moderate
Examples	Correlation, Chi-square, ANOVA	Forward, Backward, RFE	Lasso, Tree Importance
Best suited for	Many features and fast screening	Smaller feature sets	Practical balance of speed and relevance

Feature Selection: Correct Evaluation

Leakage warning: Feature selection must be learned using training data only. During cross-validation it must be repeated independently inside every training fold. A pipeline is the safest approach.

Full dataset → Feature selection → Train-test split (Wrong Approach)

For this reason:

1. The test score may appear unrealistically high.
2. The evaluation results will not reflect the model's true performance.
3. The model's performance may decrease when applied to new, unseen data.

Feature Selection: Correct Evaluation

Correct Approach:

Split data first



Learn feature selection from training data



Apply the learned selection to train and test data



Train the model on selected training features



Evaluate on selected test features

Feature Selection: Correct Evaluation

Why Must Feature Selection Occur Inside Cross-Validation?

Suppose 5-fold cross-validation is being used. In each iteration:

- Four folds are used for training.
- One fold is used for validation.

If feature selection is performed on the entire dataset before cross-validation, information from every validation fold will influence the feature-selection process.

This creates **data leakage** and may produce an unrealistically high validation score.

Feature selection must therefore be performed separately in each fold using only that fold's training portion.

This is necessary because different training folds may produce different sets of selected features.

8. Cross-Validation

Feature Selection

Retains a subset of the original features

Preserves the original feature names

Generally more interpretable

Examples: RFE, Lasso, Chi-square

Feature Extraction

Combines or transforms original features to create new dimensions

The meaning of the newly created features may not always be easy to understand

Generally less interpretable

Examples: PCA, LDA, Autoencoder

Cross-Validation

Cross-validation (CV) is a resampling method that repeatedly divides development data into training and validation portions. It estimates how consistently a model will perform on unseen data and supports model or hyperparameter selection.

K-Fold Cross-Validation:

- Divide the training/development data into K approximately equal folds.
- Train on K-1 folds and validate on the remaining fold.
- Repeat K times so every fold serves as validation once.
- Calculate the mean and standard deviation of the K validation scores.

$$CV\ mean = (score1 + score2 + \dots + scoreK) / K$$

Cross-Validation: Numerical Example

Suppose 5-fold accuracies are 0.82, 0.85, 0.80, 0.88, and 0.85.

Mean accuracy = $(0.82+0.85+0.80+0.88+0.85)/5 = 0.84 = 84\%$

A large standard deviation would indicate that performance changes considerably across folds; the model may be unstable or the dataset may be small/heterogeneous.

CV and Final Test Set

Cross-validation does not justify repeatedly checking the final test set. Use CV on development data to choose the method; then retrain on all development data and evaluate once on the untouched test set.

Pipeline rule: Imputation, scaling, encoding, feature selection, and any data-dependent transformation must be fitted separately inside each training fold.

9. Gradient Descent

Gradient Descent is an iterative optimization algorithm used to minimize the loss/cost function of a machine learning model.

Goal

$$\min J(\theta)$$

Where:

- $J(\theta)$ = Cost/Loss function
- θ = Model parameters such as weights and bias
- w or m = Weight/slope
- b = Bias/intercept

The gradient gives the direction of the **steepest increase** of the cost function. Therefore, to minimize the cost, we move in the **opposite direction** of the gradient.

Gradient Descent Formula

The general formula is:

$$\theta_{new} = \theta_{old} - \alpha \frac{\partial J}{\partial \theta}$$

For a single parameter m :

$$m_{new} = m_{old} - \alpha \frac{dJ}{dm}$$

Basic Working Process

1. Initialize Parameters
2. Make Predictions
3. Calculate Error / Cost
4. Calculate Gradient
5. Update Parameters
6. Calculate Cost Again
7. Repeat
8. Reach Minimum Cost

Step-by-Step

x	y
1	2
2	4

Suppose our prediction model is: $\hat{y} = mx$

There is no bias b in this simplified example.

Initialize the Parameter

Suppose:

$$m = 0$$

Our model is:

$$\hat{y} = mx$$

For the first data point:

$$\hat{y}_1 = 0(1) = 0$$

For the second:

$$\hat{y}_2 = 0(2) = 0$$

Therefore:

$$Actual = [2, 4]$$

$$Predicted = [0, 0]$$

The predictions are incorrect, so we need to update m .

Define the Cost Function

$$J(m) = \sum_{i=1}^n (y_i - mx_i)^2$$

For our two observations:

First observation:

$$y_1 - mx_1 = 2 - m(1) = 2 - m$$

Second observation:

$$y_2 - mx_2 = 4 - m(2) = 4 - 2m$$

Therefore:

$$J(m) = (2 - m)^2 + (4 - 2m)^2$$

At $m = 0$:

$$\begin{aligned} J(0) &= (2 - 0)^2 + (4 - 0)^2 \\ &= 4 + 16 \end{aligned}$$

$$J(0) = 20$$



So the initial cost is **20**.

Differentiate the Cost Function

Now we need to determine how changing m changes the cost.

Our cost function is:

$$J(m) = (2 - m)^2 + (4 - 2m)^2$$

Differentiate with respect to m :

$$\frac{dJ}{dm} = 2(2 - m)(-1) + 2(4 - 2m)(-2)$$

Simplify:

$$= -2(2 - m) - 4(4 - 2m)$$

Expand:

$$= -4 + 2m - 16 + 8m$$

Therefore:

$$\boxed{\frac{dJ}{dm} = 10m - 20}$$

This derivative represents the **gradient**.

Calculate the Current Gradient

We currently have:

$$m = 0$$

Therefore:

$$\frac{dJ}{dm} = 10(0) - 20$$

$$\boxed{\frac{dJ}{dm} = -20}$$

The gradient is **negative**.

This means m should move toward a larger value to reduce the cost.

Update the Parameter

Gradient Descent formula:

$$m_{new} = m_{old} - \alpha \frac{dJ}{dm}$$

Suppose the learning rate is:

$$\alpha = 0.1$$

Then:

$$m_{new} = 0 - (0.1)(-20)$$

Because subtracting a negative number becomes addition:

$$m_{new} = 0 + 2$$

Therefore:

$$m_{new} = 2$$

Gradient Descent has found the optimal slope.

Calculate the New Cost

Now:

$$m = 2$$

The model becomes:

$$\hat{y} = 2x$$

For $x = 1$:

$$\hat{y} = 2(1) = 2$$

For $x = 2$:

$$\hat{y} = 2(2) = 4$$

So:

$$Predicted = [2, 4]$$

and:

$$Actual = [2, 4]$$

Now calculate the cost:

$$\begin{aligned} J(2) &= (2 - 2)^2 + (4 - 4)^2 \\ &= 0 + 0 \end{aligned}$$

Therefore:

$$J(2) = 0$$

The model has reached the minimum cost.

Types of Gradient Descent

1. Batch Gradient Descent

Uses all training samples before performing one parameter update.

2. Stochastic Gradient Descent (SGD)

Uses one training sample per update.

3. Mini-Batch Gradient Descent

Uses a small group of samples for each update.

10. Model Evaluation Metrics

Why Evaluation Matters?

A metric translates prediction quality into a number. The correct metric depends on the task, class balance, error cost, and business objective. Training score alone is never sufficient evidence of generalization.

Regression Metrics

Term	Meaning
MAE	Average absolute error; same unit as target; comparatively robust to large errors.
MSE	Squares errors; penalizes large errors strongly; unit is squared.
RMSE	Square root of MSE; target unit; sensitive to large errors.
R-squared	Proportion of variance explained relative to predicting the mean. Best is 1; 0 means no improvement over the mean; it can be negative on test data.

Regression Example

Actual values: [3, 5, 7]; predictions: [2, 5, 9]. Errors: [1, 0, -2] if error is actual minus predicted; absolute errors: [1,0,2]; squared errors: [1,0,4].

MAE = $3/3 = 1$; MSE = $5/3 = 1.667$; RMSE approx 1.291

Classification Confusion Matrix

Term	Meaning
True Positive (TP)	Positive case correctly predicted positive.
True Negative (TN)	Negative case correctly predicted negative.
False Positive (FP)	Negative case incorrectly predicted positive; false alarm.
False Negative (FN)	Positive case incorrectly predicted negative; missed positive.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}); \quad \text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP}); \quad \text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

Classification Example

Let $TP=40$, $TN=50$, $FP=10$, $FN=20$. Total=120.

Accuracy=90/120=75 % ; Precision=40/50=80 % ; Recall=40/60=66.67 %

Specificity=50/60=83.33 % ; F1 approx 72.73 %

Choosing a Classification Metric

- Accuracy is useful when classes are reasonably balanced and error costs are similar.
- Precision matters when false positives are costly, such as incorrectly flagging legitimate email as spam.
- Recall matters when false negatives are costly, such as missing a serious disease or fraud.
- F1 balances precision and recall and is useful for imbalanced classes.
- ROC-AUC measures ranking across thresholds; PR-AUC is often more informative when the positive class is rare.
- Log loss evaluates predicted probabilities and heavily penalizes confident wrong predictions.

Macro, Micro and Weighted Averages

Term	Meaning
Macro average	Metric calculated per class, then equally averaged; highlights minority-class performance.
Micro average	Aggregates all class decisions before calculation; dominated by frequent classes.
Weighted average	Class-wise metric weighted by support; reflects distribution but can hide poor minority performance.

11. Bias and Variance

Bias:

Bias is systematic error caused by assumptions that are too simple or inappropriate. A high-bias model fails to capture the underlying relationship and usually performs poorly even on training data.

Variance:

Variance is sensitivity to changes in the training sample. A high-variance model learns noise and accidental details; it may perform extremely well on training data but poorly on unseen data.

Term	Meaning
High bias	Model too simple; strong assumptions; training and validation errors both high.
High variance	Model too flexible; training error low but validation/test error much higher.
Low bias, low variance	Desired generalizable model; both errors reasonably low and close.

Sources

- Bias increases with an overly simple model, insufficient features, excessive regularization, or insufficient training.
- Variance increases with an overly complex model, too many noisy features, very small data, leakage-prone tuning, or too little regularization.
- Noise is irreducible uncertainty in data and labels; a model cannot remove all of it.

Diagnosing with Learning Curves

- High bias: training and validation scores converge at a poor level; more data alone may help little.
- High variance: training score is high, validation score lower, and the gap is large; more representative data often helps.
- Learning curves plot performance against training-set size and help diagnose both cases.

12. Underfitting and Overfitting

Underfitting

Underfitting occurs when a model is too simple or insufficiently trained to learn important patterns. It performs poorly on both training and unseen data. It is closely associated with high bias.

Overfitting

Overfitting occurs when a model learns training-specific noise and details that do not generalize. It has very low training error but substantially higher validation/test error. It is closely associated with high variance.

Term	Meaning
Underfitting signs	High training error; high validation error; small gap; poor predictions everywhere.
Good fit signs	Low, similar training and validation errors; stable CV scores.
Overfitting signs	Very low training error; noticeably higher validation error; large gap; unstable folds.

Causes and Remedies

Term	Meaning
Underfitting remedies	Use a more expressive model; add informative features; reduce excessive regularization; train longer; tune optimization.
Overfitting remedies	Simplify the model; regularize; select features; gather more data; use augmentation; prune trees; apply dropout/early stopping; use CV.

Model Complexity View

As complexity increases, training error normally decreases. Validation error initially decreases because the model learns useful patterns, reaches a minimum, and then rises when the model begins fitting noise. The best complexity lies near the minimum validation error, not necessarily at the minimum training error.

Data Leakage vs Overfitting

Both can produce unrealistically strong evaluation results, but they differ.

Overfitting learns training noise; leakage gives the model information that should not be available. Leakage must be corrected in data design, while overfitting is controlled through model complexity, regularization, and validation.

13. Bias-Variance Trade-off

The bias-variance trade-off is the need to balance a model that is simple enough to remain stable with one that is flexible enough to learn the true pattern. Increasing complexity generally reduces bias but may increase variance; decreasing complexity generally reduces variance but may increase bias.

Simple model -> higher bias, lower variance

Complex model -> lower bias, higher variance

How to Find the Balance

- Choose a meaningful evaluation metric.
- Keep a final test set untouched.
- Compare complexity or hyperparameters using cross-validation.
- Use regularization or early stopping where appropriate.
- Inspect training-validation gaps and learning curves.
- Select the simplest model whose validated performance satisfies the objective.
- Retrain on all development data and evaluate once on the test set.

Effects of Common Actions

Term	Meaning
Increase model complexity	Usually decreases bias and increases variance.
Increase regularization	Usually increases bias and decreases variance.
Add useful training data	Often reduces variance without increasing bias significantly.
Add irrelevant features	Can increase variance and overfitting.
Feature engineering	Can reduce bias if it exposes the true relationship.
Bagging / Random Forest	Often reduces variance.
Boosting	Often reduces bias, but may overfit if pushed too far.
Early stopping	Limits effective complexity and can reduce variance.

Term	Meaning
Machine learning	Learns patterns from data to improve a task.
Supervised	Uses labeled input-output examples.
Unsupervised	Discovers structure from unlabeled data.
Reinforcement	Learns a policy from rewards through interaction.
Feature / label	Input variable / target output.
Train / validation / test	Fit parameters / tune choices / final estimate.
Preprocessing	Cleans and converts raw data.
Feature engineering	Creates more informative representations.
Feature selection	Keeps useful original features.
Cross-validation	Repeated resampling for reliable validation.
Gradient descent	Moves parameters opposite the gradient to reduce cost.
Bias	Systematic error; model too simple.
Variance	Sensitivity to training sample; model too unstable.
Underfitting	High train and validation errors.
Overfitting	Low train error, higher validation/test error.
Trade-off	Balance flexibility and stability for generalization.